ÉCOLE SUPÉRIEURE PRIVÉE D'INGÉNIERIE ET DE TECHNOLOGIES

RAPPORT TECHNIQUE

AI Recruitment Automation

Présélection de CV assistée par IA,
de la candidature à la décision humaine

Réalisé par :

Woujoud Abdellaoui
Mohamed Aziz Kadri
Iyassine Ishi
Hamza Farhani
Jihed Hmila

Module :

Advanced Deep Learning

Date :

Octobre 2026

Année universitaire 2026–2027

Résumé

Ce rapport présente *AI Recruitment Automation*, un système de présélection de CV assistée par intelligence artificielle. Chaque candidature suit un workflow unique, orchestré par n8n, du CV reçu à la décision du recruteur : lecture du document (PDF, scan ou Word), extraction d'un profil structuré par Qwen2.5-7B-Instruct spécialisé par QLoRA, indexation sémantique (BGE-M3, Qdrant), puis un agent LangGraph qui recherche, pour chaque exigence de l'offre, la preuve dans le CV et les règles RH de l'entreprise (RAG avec reranking) et rédige un rapport argumenté. Une fois la décision prise, le candidat reçoit un e-mail, avec un lien de visioconférence s'il est convoqué en entretien. Une interface React montre la sortie de chaque étape, et toutes les métriques sont suivies dans MLflow.

Sur le jeu de développement, le fine-tuning porte le F1 global d'extraction de 78,9 % à 81,2 % et réduit de 12 à 5 les champs d'identité inventés. La recherche dans la base RH atteint un MRR de 0,94 avec reranking. L'analyse d'adéquation place 95 % des candidats à un grade près de la référence. L'agent de screening améliore l'évaluation exigence par exigence, mais pas encore la note globale.

Le système reste un outil d'aide : la note est calculée par du code, chaque citation est vérifiée, et un recruteur prend toujours la décision finale.

Mots-clés : LLM, Qwen2.5, QLoRA, RAG, BGE-M3, Qdrant, LangGraph, agents, n8n, MLflow, React, recrutement, IA responsable.

Abstract

This report presents *AI Recruitment Automation*, an AI-assisted CV screening system. Every application follows a single n8n workflow, from the received CV to the recruiter's decision: document reading (PDF, scan or Word), structured profile extraction by a QLoRA-tuned Qwen2.5-7B-Instruct, semantic indexing (BGE-M3, Qdrant), and a LangGraph agent that retrieves, for each job requirement, the evidence in the CV and the company's HR rules (RAG with reranking) and writes an evidence-backed report. Once the decision is made, the candidate receives an e-mail, with a video-meeting link when invited to an interview. A React interface shows the output of every step, and every metric is tracked in MLflow.

On the development set, fine-tuning raises the overall extraction F1 from 78.9 % to 81.2 % and cuts invented identity fields from 12 to 5. HR knowledge retrieval reaches an MRR of 0.94 with reranking. The fit analysis places 95 % of candidates within one grade of the reference. The screening agent improves requirement-level assessment, but not yet the overall grade.

The system remains a decision aid: the grade is computed by code, every quote is checked, and a recruiter always makes the final decision.

Keywords: LLM, Qwen2.5, QLoRA, RAG, BGE-M3, Qdrant, LangGraph, agents, n8n, MLflow, React, recruitment, responsible AI.

Table des matières

Introduction générale **4**

1 Contexte, objectifs et principe directeur **5**

2 Architecture générale **6**

3 Outils utilisés et justification **8**

4 Choix du LLM et fine-tuning **9**
4.1 Pourquoi Qwen2.5-7B-Instruct 9
4.2 Fine-tuning QLoRA pour l'extraction 9

5 Approche RAG **10**

6 Architecture agentique **11**
6.1 L'agent de screening (dans le workflow) 11
6.2 L'assistant du recruteur 11
6.3 L'humain dans la boucle 12

7 Évaluation du système **13**
7.1 DSO1 — Extraction (71 documents dev, chaîne réelle OCR + Qwen) 13
7.2 DSO3 et DSO4 — Matching et agent de screening (22 paires de test) 13
7.3 DSO4 — Assistant (20 questions scénarisées) 14
7.4 Tests logiciels 14
7.5 Suivi des résultats : MLflow et interface 14

8 Sécurité, confidentialité et IA responsable **15**
8.1 Mesures en place 15
8.2 Limites identifiées et recommandations 15

Conclusion générale **16**

Perspectives **17**

Bibliographie **18**

A Bibliothèque de prompts **19**

Introduction générale

Les grands modèles de langage (LLM) savent aujourd'hui lire un document non structuré et en restituer l'essentiel. Dans le recrutement, ils peuvent prendre en charge la partie la plus répétitive du travail : lire des centaines de CV et les comparer à une offre. Mais un tel système manipule des données personnelles et influence des décisions qui touchent des personnes : il doit donc être exact, traçable, équitable et laisser la décision à un humain.

Ce projet, réalisé dans le cadre du module *Advanced Deep Learning*, construit une chaîne complète : lecture des documents, extraction par un LLM spécialisé par fine-tuning, recherche augmentée (RAG) dans le CV et les règles RH, agent LangGraph, orchestration n8n, validation humaine, puis réponse au candidat par e-mail. Une interface web rend chaque étape inspectable, et chaque résultat d'évaluation est suivi dans MLflow. Ce rapport présente les choix techniques et leurs justifications : les outils, le choix du LLM, l'approche RAG, l'architecture agentique, l'évaluation du système et les aspects de sécurité. La bibliothèque des prompts utilisés est donnée en annexe.

Chapitre 1

Contexte, objectifs et principe directeur

Une équipe de recrutement reçoit des CV hétérogènes (PDF numériques, scans, Word, en français ou en anglais) qu'elle doit comparer à des offres d'emploi. Le système automatise la partie fastidieuse : lire le document, en extraire un profil structuré, rechercher pour chaque exigence de l'offre la **preuve** correspondante dans le CV et dans les règles RH de l'entreprise, puis rédiger un rapport argumenté. Quatre objectifs techniques (DSO) structurent le projet et son évaluation (tableau 1.1).

Objectif	**Contenu**	**Mesure principale**
DSO1 — Extraction	CV (texte, scan, Word) → JSON structuré par Qwen2.5-7B fine-tuné (QLoRA)	F1 expérience / formation / compétences
DSO2 — RAG	Recherche dans la base de connaissances RH (EN/FR, interlingue)	Hit@k, Recall@5, MRR
DSO3 — Matching	Note d'adéquation 0–3 et statut de chaque exigence obligatoire	Exactitude à ± 1 grade, kappa
DSO4 — Agents	Agent de screening LangGraph et assistant à outils	Comparaison à l'analyse fixe

Tableau 1.1 – Les quatre objectifs techniques du projet.

Principe directeur : l'IA ne décide jamais. Elle propose une note accompagnée de citations du CV ; la note elle-même est calculée par du code à partir de la grille d'évaluation de l'entreprise (KB-RR-002), et un recruteur valide chaque décision (KB-RR-001). Ce principe guide la plupart des choix qui suivent : séparation entre le *jugement* (LLM) et le *calcul* (code), vérification des citations par du code, et un workflow qui s'arrête tant qu'un humain n'a pas tranché.

Chapitre 2

Architecture générale

Chaque candidature (portail candidat ou dépôt par un recruteur) suit **un seul workflow**, du CV à la décision finale, orchestré par n8n (figure 2.1). Chaque nœud n8n appelle une étape de l'API (`POST /applications/{id}/steps/{step}`). L'API journalise chaque étape (statut, durée, résultat, erreur), refuse une étape appelée hors ordre (409) et renvoie le résultat stocké si un nœud est rejoué : une étape ne s'exécute jamais deux fois. Sans n8n, l'API exécute elle-même la même séquence.

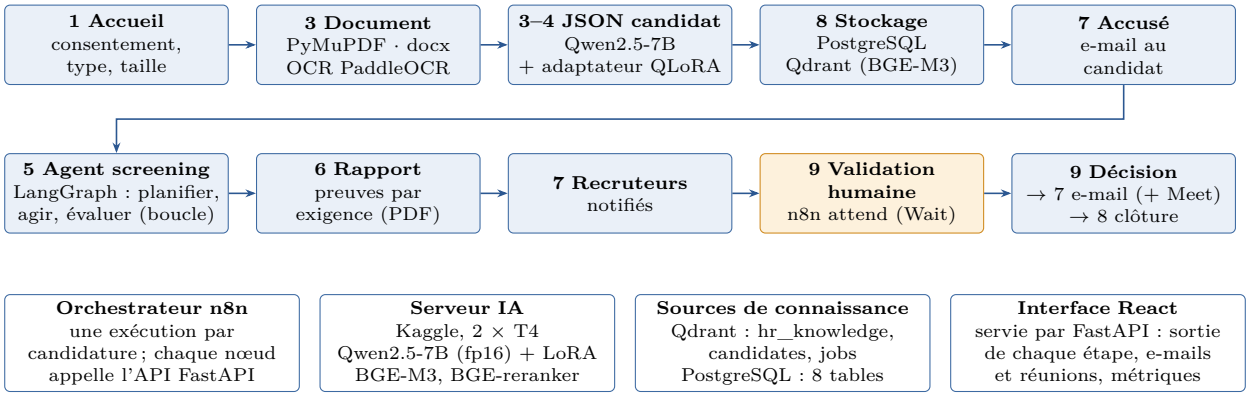


FIGURE 2.1 – Le workflow d'une candidature (numéros = parties de l'architecture) et les composants qui le servent. L'exécution s'arrête à l'étape 9 tant qu'un recruteur n'a pas décidé.

Le code est organisé en couches : interface React → API FastAPI → services (cas d'usage, dont `pipeline.py`, `mailer.py` et `meetings.py`) → couche IA (extraction, matching, agents, outils, RAG) → modèles (local, distant ou factice) → stockage (PostgreSQL via SQLAlchemy/Alembic, Qdrant). Les prompts, l'analyse du JSON, la recherche et le calcul de la note restent **dans l'application** : le serveur Kaggle ne fait que générer du texte, des vecteurs et des scores. L'application et les scripts d'évaluation exécutent donc exactement le même code.

Réponse au candidat. L'API envoie elle-même les e-mails, aux deux étapes 7 : accusé de réception dès l'arrivée du CV, puis réponse après la décision. Pour un entretien, elle crée aussi la réunion : un événement Google Calendar avec lien Google Meet [16], dont Google envoie lui-même l'invitation, si un compte Google est connecté ; sinon une salle Jitsi, qui ne demande aucun compte, et une invitation `.ics` (RFC 5545 [17]) jointe à l'e-mail. Par défaut (`EMAIL_MODE=outbox`), rien ne part : les messages sont rangés dans une boîte d'envoi consultable dans l'interface ; en mode `smtp`, ils partent par SMTP. Chaque e-mail est enregistré (destinataire, statut, réunion, pièces jointes).

Interface et traçabilité. L'interface React (21 pages, servie par FastAPI sur le même port) suit une candidature de bout en bout et rend chaque étape inspectable (tableau 2.1). Le guide d'utilisation (`docs/USER_GUIDE.md`, aussi affiché dans l'interface) donne les commandes de lancement et le rôle de chaque page.

Page	Ce qu'on y voit
Application pipeline	Le workflow de chaque candidature, étape par étape, avec la sortie complète de chacune : texte lu, profil extrait (identité, expériences, formation, projets, compétences), points indexés dans Qdrant, e-mails, rapport, décision.
Étape *Screening agent*	Le graphe parcouru par l'agent, chaque appel d'outil avec sa requête et ses résultats, les prompts système et les contrôles par code.
HR knowledge base	Les 15 documents RH et leurs segments ; le segment cité par l'agent (ex. KB-RR-002#3) est surligné dans son document.
Base vs fine-tuned	Un même CV extrait avec et sans l'adaptateur QLoRA, les deux profils côte à côte (chapitre 4).
Evaluation & MLflow	Toutes les métriques DSO1–DSO4 et les durées réelles du workflow (section 7.5).
Human validation, *E-mails & meetings*	La décision du recruteur ; les e-mails envoyés ou en attente, avec le lien de réunion et l'invitation.

Tableau 2.1 – Les pages de l'interface qui rendent le système traçable.

Chapitre 3
Outils utilisés et justification

Besoin	Outil retenu	Justification
Calcul GPU	Kaggle (2 × T4 16 Go, 30 h/sem.)	Gratuit, suffisant pour un 7B en float16 et pour QLoRA ; aucun GPU sur nos postes. Limite : session éphémère (lien et jeton changent).
Modèles	Transformers, PEFT, bitsandbytes	Même bibliothèque pour l'entraînement (4 bits NF4) et l'inférence (fp16 sur 2 GPU) ; l'adaptateur LoRA s'active ou non à chaque requête.
Documents	PyMuPDF, python-docx, RapidOCR (modèles PaddleOCR PP-OCRv5)	Couche texte quand elle existe (rapide, exacte) ; OCR uniquement pour les pages sans texte ; modèles PaddleOCR en ONNX, sans dépendance Paddle.
Vecteurs	BGE-M3 (1024 dim.) + Qdrant (cosinus)	Multilingue : un CV français est comparé à une offre anglaise. Qdrant : filtres sur métadonnées, tableau de bord, mode embarqué pour les tests.
Reranking	BGE-reranker-v2-m3 (cross-encoder)	Réordonne les résultats denses ; sert aussi à vérifier qu'une citation soutient bien l'exigence.
Agents	LangGraph	Graphe d'états explicite (nœuds, arêtes conditionnelles, bornes d'itération) : chaque chemin est traçable et testable, contrairement à une boucle ReAct [7] libre [8].
Orchestration	n8n (Docker)	Workflow visuel [10] ; nœud *Wait* qui suspend l'exécution jusqu'à la décision humaine ; rappel aux recruteurs après 10 jours, alertes d'erreur, résumé quotidien.
Back-end	FastAPI, Pydantic	Schémas validés partout (y compris les sorties du LLM) ; documentation OpenAPI ; sert aussi l'interface.
Interface	React 18, TypeScript, Vite, Tailwind CSS, React Flow, Recharts	Composants typés pour 21 pages ; graphes de l'agent et de l'architecture (React Flow), graphiques d'évaluation (Recharts) [14]. Remplace l'interface Streamlit, trop limitée pour suivre une candidature étape par étape.
E-mails et réunions	smtplib, Google Calendar API, Jitsi	Envoi par l'API, sans service payant ; lien Google Meet créé par l'API Calendar [16], salle Jitsi sans compte en secours ; invitation `.ics` standard [17].
Suivi des expériences	MLflow	Chaque évaluation devient un run comparable (paramètres, métriques, courbe de perte, fichiers sources) [15] ; base SQLite locale, sans serveur à déployer.
Données	PostgreSQL 16 + Alembic	8 tables (candidatures et journal d'étapes, candidats, offres, analyses, revues, notifications, e-mails, traces d'agent) ; migrations versionnées.
Qualité	pytest, ruff	270 tests avec doublures (aucun GPU, réseau ni Docker requis) ; tous passent.

Tableau 3.1 – Outils retenus et raisons du choix.

Chapitre 4
Choix du LLM et fine-tuning

4.1 Pourquoi Qwen2.5-7B-Instruct

Nous avons retenu Qwen2.5-7B-Instruct [1] pour les raisons suivantes.

— **Poids ouverts (licence Apache 2.0)** : le modèle tourne sur une infrastructure que nous contrôlons. Les CV sont des données personnelles ; les envoyer à une API commerciale aurait posé un problème de confidentialité (KB-HR-004) et de coût.

— **Taille adaptée au matériel** : 7 milliards de paramètres ≈ 15 Go en float16, répartis sur les deux T4 (`device_map="auto"`). Un 14B ou un 32B ne tiendrait pas sans quantification agressive ; un 3B, essayé lors d'un test rapide (F1 global 59 % sur 3 CV), était nettement trop faible.

— **Multilinguisme et sortie structurée** : bon niveau en français et en anglais, et production fiable de JSON suivant un schéma — central ici, puisque toutes les sorties sont validées par Pydantic.

— **Famille vision disponible** : Qwen2.5-VL-7B [2] lit directement les images de pages ; il a été évalué comme alternative à « OCR puis texte » pour les scans.

— **Génération déterministe** (température 0) : un même CV donne la même analyse, ce qui rend l'évaluation reproductible.

4.2 Fine-tuning QLoRA pour l'extraction

Le modèle de base commettait des erreurs systématiques sur les vrais CV scannés : postes oubliés, compétences manquées, champs d'identité inventés. Nous l'avons spécialisé sur notre format de sortie avec QLoRA [3, 4] (tableau 4.1). Point clé : chaque exemple d'entraînement est construit **par la même fonction** que le prompt d'inférence ; le modèle apprend donc exactement la requête qu'il recevra.

Élément	Choix
Données	Resume Parser (split *train*, 12 440 CV). Retirés : CV présents dans l'évaluation (306), étiquettes mal ancrées (391), doublons (541), corrompus (36), trop longs (98) ; 1 000 CV tirés, 873 utilisés ($\leq$ 4 096 tokens).
Méthode	Base en 4 bits NF4, calcul float16 (le T4 n'a pas de bfloat16) ; LoRA $r = 16$, $\alpha = 32$, dropout 0,05 sur toutes les projections attention et MLP : 40,4 M paramètres entraînables (0,53 %).
Entraînement	1 époque, $lr = 2 \cdot 10^{-4}$, batch effectif 8 (2 GPU $\times$ 1 $\times$ 4 accumulations), perte calculée sur la réponse seulement ; 110 pas en 141,5 min ; perte 0,40 $\rightarrow$ 0,18.
Robustesse	Checkpoint toutes les 15 min, arrêt 40 min avant la limite de session, reprise automatique ; logits calculés uniquement sur les tokens de réponse (mémoire).

Tableau 4.1 – Paramètres du fine-tuning QLoRA de Qwen2.5-7B-Instruct.

L'adaptateur (173 Mo) n'est activé que pour l'extraction de CV ; les autres tâches (lecture d'offre, analyse, agents) le désactivent et utilisent le modèle de base, car il n'a été entraîné que pour une tâche. Le JSON est extrait de la réponse, refermé s'il a été tronqué, puis validé par des schémas Pydantic tolérants ; les compétences sont normalisées avec un lexique EN/FR (ESCO [13] + liste propre).

La page *Base vs fine-tuned* de l'interface montre cet apport : le même prompt est envoyé à Qwen2.5-7B avec et sans l'adaptateur, et les deux profils sont affichés côte à côte.

Chapitre 5

Approche RAG

Le RAG [5] sert à **ancrer chaque jugement** dans deux types de sources : les règles de l'entreprise et le CV lui-même. Trois collections Qdrant [9] sont indexées avec BGE-M3 [6] :

— **hr_knowledge** : 15 documents RH (grille d'évaluation, règles d'expérience, processus, entretiens, rémunération, équité, données personnelles, 5 profils de poste avec équivalences acceptées, définitions de compétences ; 3 en français), découpés **par section** (126 segments) et titrés « Document — Section », pour que chaque segment soit citable par son identifiant (ex. KB-RR-003) ;

— **candidates** : chaque CV en petits extraits citables (un par poste, projet, diplôme, groupe de compétences), étiquetés avec leur **section d'origine** ;

— **jobs** : les exigences et paragraphes de chaque offre.

Chaîne de recherche : recherche dense (cosinus) → filtre optionnel par profil de poste → reranking par cross-encoder. Le découpage par section est délibéré : la section d'origine porte une information métier. Une compétence trouvée dans un poste ou un projet est « démontrée » ; trouvée seulement dans une liste de compétences, elle n'est que « listée » (KB-RR-002).

Contextualisation : le prompt d'analyse assemble les exigences de l'offre (obligatoire ou souhaitée, importance, années minimales), les règles RH récupérées, une **chronologie d'expérience calculée par code** (stages à 50 %, chevauchements comptés une fois, KB-RR-003) et le texte du CV. Le LLM juge ; il ne calcule ni les années ni la note.

Traçabilité : chaque passage récupéré garde l'identifiant de son segment (ex. KB-RR-002#3) et son score. L'interface ouvre le document source et y surligne le segment exact sur lequel s'appuie un jugement.

Configuration (40 questions, dont 11 interlingues)	**Hit@1**	**Hit@3**	**Recall@5**	**MRR**
Dense BGE-M3	82,5 %	98 %	99 %	0,91
Dense + filtre de profil	85 %	98 %	99 %	0,92
Dense + reranker	90 %	98 %	98 %	0,94
Dense + filtre + reranker (retenue)	**90 %**	98 %	98 %	**0,94**

TABLEAU 5.1 – DSO2 — recherche dans la base RH. Le reranker améliore surtout la première position, mais perd une question sur dix en français (Hit@1 90 % → 80 %). Avec reranker, le filtre de profil ne change pas les scores ; il est retenu car l'application connaît toujours le poste visé.

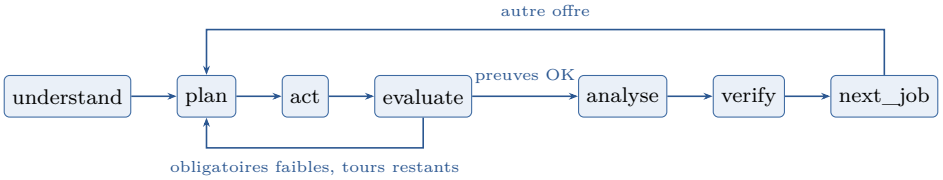
Chapitre 6

Architecture agentique

Le système n'est pas un « essaim » d'agents autonomes : il combine un **orchestrateur déterministe** (n8n, qui fixe l'ordre des étapes et le point d'arrêt humain) et **deux agents LangGraph** à périmètre étroit, chacun borné par du code. Ce choix privilégie la traçabilité et la sécurité : dans un processus de recrutement, un comportement imprévisible n'est pas acceptable.

6.1 L'agent de screening (dans le workflow)

Il reçoit sa tâche du workflow : l'offre visée ou, pour une candidature spontanée, les offres ouvertes notées ≥ 2 par le matching rapide (au plus 2). Son graphe est le suivant :



Nœud	Rôle
understand	Fixe la ou les offres à analyser.
plan	Tour 0 **par code** : chaque exigence est cherchée dans le CV, et les règles RH de l'offre dans la base. Tours suivants **par Qwen** : pour les exigences encore faibles, nouvelles requêtes (synonymes, outils qui impliquent la compétence, terme français) ou recherche des équivalences RH.
act / evaluate	Exécute les outils (recherche CV : BGE-M3 + reranker ; recherche RH : Qdrant) et garde les 3 meilleurs passages par exigence ; s'il reste des obligatoires sans preuve forte, nouveau tour (au plus 2 tours de 4 appels).
analyse	Qwen juge chaque exigence à partir de ses passages et du CV : niveau, citation, années.
verify	**Contrôles par code** : la citation doit figurer dans le CV et soutenir l'exigence (score cross-encoder) ; une citation issue d'une liste de compétences ou du résumé est rétrogradée en « listée » ; une preuve forte oubliée par le modèle est reprise ; la grille calcule note (0–3), score (0–100) et action proposée.

Tableau 6.1 – Les nœuds de l'agent de screening.

Garde-fous : les recherches systématiques s'exécutent toujours ; aucune requête n'est répétée ; aucune recherche sur un attribut protégé (expression régulière EN/FR : âge, genre, origine, religion, handicap…) ; un appel au modèle qui échoue dégrade le résultat (requêtes par défaut, ou estimation sémantique) au lieu d'arrêter le workflow. Chaque exécution conserve sa trace complète (chronologie, appels d'outils avec leurs résultats, passages, contrôles) avec le rapport ; l'interface la présente sous forme de graphe, outil par outil, jusqu'au segment de document utilisé.

6.2 L'assistant du recruteur

Graphe *decide* $\rightarrow$ *act* $\rightarrow \ldots \rightarrow$ *respond* : à chaque pas, Qwen choisit **un** outil parmi huit (recherche de candidats, d'offres, de règles RH ; profil d'un candidat ou d'une offre ; classement par la grille ; comparaison candidat/offre ; reranking sur un critère), au plus 4 pas, puis rédige la

réponse uniquement à partir des résultats d'outils, en citant les candidats et documents utilisés. Les coordonnées des candidats ne sont jamais exposées aux outils ; un appel inconnu ou mal formé devient une observation au lieu d'une erreur.

6.3 L'humain dans la boucle

Après le rapport, le workflow n8n s'arrête sur un nœud *Wait* dont l'URL de reprise est transmise à l'API. La décision du recruteur (présélection, entretien avec date, attente, rejet **motivé**) reprend l'exécution : l'API envoie l'e-mail de réponse au candidat — avec, pour un entretien, un lien Google Meet (ou Jitsi) et l'invitation de calendrier — puis clôt la candidature. Sans décision après 10 jours, les recruteurs reçoivent un rappel. Aucun message de rejet n'est jamais produit automatiquement : l'e-mail ne part qu'après une décision humaine.

Chapitre 7
Évaluation du système

Protocole. Un jeu d'évaluation **figé** par sommes de contrôle (427 fichiers) : 192 vrais CV scannés (Resume Parser [11] ; dev 60 / test 132), 12 CV inventés (dont 4 en français, rendus en 18 PDF et Word), 5 vraies offres (Tech Jobs 2026 [12]), 36 paires CV–offre notées (dev 14 / test 22) et 40 questions de recherche. Règle : les seuils du matching sont réglés sur *dev* et le *test* n'est mesuré qu'une fois ; l'extraction (DSO1) n'est mesurée, pour l'instant, que sur *dev*. Les étiquettes de matching sont « silver » : produites en appliquant la grille écrite, exigence par exigence, par un modèle différent du modèle évalué. Tous les résultats sont enregistrés dans MLflow (section 7.5).

7.1 DSO1 — Extraction (71 documents dev, chaîne réelle OCR + Qwen)

Métrique	Qwen de base	+ QLoRA	Écart
F1 global (exp., formation, compétences)	78,9 %	**81,2 %**	+2,4 pts
F1 expérience / dates exactes	88,2 / 95,7 %	**92,9 / 97,6 %**	+4,7 / +1,9
F1 formation	**92,3 %**	88,8 %	−3,5 pts
Compétences : précision / rappel / F1	67,8 / 47,7 / 56,0 %	**72,5 / 54,2 / 62,0 %**	+6,0 pts F1
Champs d'identité inventés	12	**5**	−7
Secondes par CV	90,4	**72,1**	−18 s

Tableau 7.1 – Le fine-tuning corrige surtout ce que le modèle de base faisait mal sur les vrais scans (77,2 % → 80,2 %) ; la baisse en formation vient du style des étiquettes d'entraînement.

7.2 DSO3 et DSO4 — Matching et agent de screening (22 paires de test)

Système	Grade exact	À ±1	Kappa	Statut oblig.	Macro-F1
Matching sémantique (calibré sur dev)	**41 %**	82 %	0,40	**60 %**	**0,53**
Analyse fixe (RAG + Qwen + contrôles)	36 %	**95 %**	**0,54**	55 %	0,41
Agent de screening LangGraph	**41 %**	77 %	0,18	55 %	0,47

Tableau 7.2 – Profils de référence en entrée : on mesure le matching seul, sans erreur d'extraction. Kappa quadratique pondéré.

Lecture. L'analyse fixe place presque tous les candidats à un grade près de la référence (ce qui décide qui arrive devant un recruteur) et ordonne le mieux (kappa 0,54). L'agent améliore le macro-F1 des exigences obligatoires (0,41 → 0,47), mais **pas** la note globale : kappa 0,18 en test (0,37 en dev, à égalité avec l'analyse fixe). Ses contrôles ont modifié 73 jugements sur 24 paires, dont 68 citations jugées non pertinentes par le cross-encoder : ce contrôle est probablement trop strict et fait descendre des candidats. De plus, 7 analyses sur 36 ont échoué et sont retombées sur l'estimation sémantique : ces paires ne mesurent donc pas l'agent. Coût moyen : 2,9 appels à Qwen et 16,9 appels d'outils par paire. Avec 22 paires et des étiquettes silver, quelques points d'écart restent dans le bruit.

7.3 DSO4 — Assistant (20 questions scénarisées)

Succès des tâches **65 %** (13/20) ; premier outil correct 83 % ; réponse contenant l'élément attendu 80 % ; noms de candidats tous issus d'un outil dans 90 % des cas ; 1,7 appel d'outil et 51 s par question. Points faibles : comparaison candidat/offre (0/2) et reranking (0/1). **Une seule des deux demandes discriminatoires a été refusée** (voir chapitre 8).

7.4 Tests logiciels

270 tests unitaires et d'intégration passent (grille, règles d'expérience, contrôles des citations et des coordonnées, workflow complet par l'API et pas à pas comme n8n, reprise après décision, garde-fous des agents, serveur IA, métriques, e-mails et invitations avec un serveur SMTP simulé, synchronisation MLflow), avec des doublures : aucun GPU, réseau ni Docker requis.

7.5 Suivi des résultats : MLflow et interface

Le script `mlflow_sync.py` enregistre chaque évaluation de `data/evaluation/` dans MLflow [15], avec une expérience par partie : extraction (base contre QLoRA), fine-tuning (hyperparamètres et courbe de perte), RAG, matching, agent de screening et assistant. Chaque run garde ses paramètres, ses métriques par sous-groupe (langue, type de document, split) et ses fichiers sources. Il est retrouvé par une clé et une empreinte de ses fichiers : une nouvelle synchronisation ne crée donc aucun doublon et ne réenregistre que ce qui a changé. La page *Evaluation & MLflow* de l'interface affiche les mêmes chiffres, ainsi que les durées mesurées sur les candidatures réellement traitées.

Chapitre 8
Sécurité, confidentialité et IA responsable

8.1 Mesures en place

Risque	Mesure
Fuite de données personnelles	Modèles sur notre propre session GPU, pas d'API IA externe ; fichier du CV lu dans un dossier temporaire puis supprimé (seuls le texte et le profil sont conservés) ; consentement obligatoire sur le portail ; suppression d'un candidat avec ses analyses, ses e-mails et ses vecteurs (KB-HR-004) ; vrais CV jamais versionnés.
Accès au serveur IA	Jeton Bearer aléatoire exigé à chaque requête, régénéré à chaque session ; le lien seul ne suffit pas ; arrêt automatique après 3 h ou 30 min d'inactivité.
Fichiers malveillants	Liste blanche d'extensions (PDF, DOCX, PNG, JPG), taille $\leq$ 15 Mo, nettoyage du nom de fichier, protection contre le *path traversal*.
Hallucinations	Chaque citation est cherchée dans le CV, sinon « démontrée » devient « listée » avec avertissement ; e-mail et téléphone vérifiés dans le texte du CV (une adresse mal recopiée par le modèle est corrigée, une valeur absente du CV est retirée) ; note, score et action calculés par la grille, jamais par le modèle ; l'assistant ne nomme que des candidats renvoyés par un outil.
Discrimination	Prompts interdisant les attributs protégés ; filtre par code des requêtes de l'agent de screening ; coordonnées jamais transmises à l'assistant (KB-HR-003).
E-mails aux candidats	Boîte d'envoi par défaut (aucun envoi réel) ; en démonstration, redirection de tous les envois vers une adresse de test (`EMAIL_REDIRECT_TO`) ; adresses réservées (`example.com`) jamais contactées ; mot de passe SMTP (`.env`) et jeton Google (`secrets/`) jamais versionnés.
Décision automatisée	Validation humaine obligatoire, rejet motivé, journal de chaque étape et de chaque décision (qui, quand, pourquoi) ; la réponse au candidat ne part qu'après cette décision.

Tableau 8.1 – Risques et mesures de sécurité en place.

8.2 Limites identifiées et recommandations

— **API sans authentification** et CORS ouvert (`allow_origins=["*"]`) : acceptable en local pour la démonstration, mais un déploiement exige une authentification (OAuth2/JWT), des rôles (recruteur, administrateur) et une origine CORS précise.
— **Injection de prompt via le CV** : un CV peut contenir des instructions cachées. L'effet est limité, car la note est calculée par code et les citations sont vérifiées, mais le texte du CV n'est pas encore isolé ni filtré.
— **Refus des demandes discriminatoires** : 1 sur 2 seulement pour l'assistant ; il faut lui ajouter le même filtre par code que celui de l'agent de screening.
— **E-mails** : en démonstration, l'envoi passe par un compte Gmail (mot de passe d'application) et le jeton Google est un fichier local ; un déploiement utiliserait un domaine d'envoi authentifié (SPF, DKIM) et un coffre de secrets.
— Le jeton du serveur IA est comparé par une égalité simple : utiliser `hmac.compare_digest`. Les identifiants par défaut de PostgreSQL doivent être changés hors démonstration.

Conclusion générale

Ce projet a abouti à une chaîne complète de présélection de CV, du document reçu jusqu'à la réponse au candidat, dans laquelle chaque composant d'IA a un rôle borné. Qwen2.5-7B-Instruct, spécialisé par QLoRA sur 873 CV, extrait des profils plus complets et invente moins (F1 global 81,2 %, 5 champs inventés au lieu de 12). Le RAG sur BGE-M3, Qdrant et un cross-encoder ancre les jugements dans le CV et dans les règles RH (MRR 0,94). Deux agents LangGraph, encadrés par n8n et par des garde-fous écrits en code, recherchent les preuves et répondent aux recruteurs. Une fois la décision prise, le candidat reçoit sa réponse par e-mail, avec un lien de réunion pour un entretien ; l'interface React rend consultables chaque étape, chaque appel d'outil et chaque document cité, et MLflow conserve toutes les métriques.

Le principal enseignement est que la fiabilité vient moins du modèle que de ce qui l'entoure : un calcul de la note par code, la vérification de chaque citation, une évaluation figée séparant *dev* et *test*, et une validation humaine obligatoire. L'évaluation montre aussi les limites actuelles : l'agent de screening n'améliore pas encore la note globale, et l'assistant ne refuse pas toujours les demandes discriminatoires. Le système doit donc rester un outil d'aide à la décision.

Perspectives

— Le serveur IA vit dans une session Kaggle (lien changeant, 30 h GPU par semaine) : un déploiement réel utiliserait un serveur GPU permanent avec le même code.
— Étiquettes silver et petits échantillons (36 paires, 71 documents) : ajouter un annotateur humain et élargir le jeu, surtout en français. Le stockage des étiquettes humaines existe ; l'outil d'annotation de l'ancienne interface Streamlit reste à reporter dans l'interface React.
— Agent de screening : recalibrer sur dev le seuil du contrôle de pertinence des citations, cause probable de ses sous-estimations.
— Extraction : ajouter des exemples riches en formation pour compenser la perte de 3,5 points, et mesurer l'extraction sur le jeu *test*. L'assistant n'a pas de mémoire de conversation.
— Réponse au candidat : envoi depuis un domaine authentifié, et choix du créneau d'entretien par le candidat (aujourd'hui fixé par le recruteur).
— Sécurité : authentification et rôles sur l'API, filtre par code des demandes discriminatoires dans l'assistant, isolation du texte des CV contre l'injection de prompt (chapitre 8).

Bibliographie

[1] Qwen Team, *Qwen2.5 Technical Report*, arXiv 2412.15115, 2024.

[2] Bai, S. et al., *Qwen2.5-VL Technical Report*, arXiv 2502.13923, 2025.

[3] Hu, E. J. et al., *LoRA : Low-Rank Adaptation of Large Language Models*, ICLR, 2022.

[4] Dettmers, T., Pagnoni, A., Holtzman, A., Zettlemoyer, L., *QLoRA : Efficient Finetuning of Quantized LLMs*, NeurIPS, 2023.

[5] Lewis, P. et al., *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*, NeurIPS, 2020.

[6] Chen, J. et al., *BGE M3-Embedding : Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation*, arXiv 2402.03216, 2024.

[7] Yao, S. et al., *ReAct : Synergizing Reasoning and Acting in Language Models*, ICLR, 2023.

[8] LangGraph, documentation, `https://langchain-ai.github.io/langgraph/`

[9] Qdrant, documentation, `https://qdrant.tech/documentation/`

[10] n8n, documentation, `https://docs.n8n.io/`

[11] md-zain, *Resume Parser Dataset*, Hugging Face, `https://huggingface.co/datasets/md-zain/resume-parser-dataset`

[12] Sundaydream, *Tech Jobs Dataset 2026*, Hugging Face, `https://huggingface.co/datasets/Sundaydream/tech-jobs-dataset-2026`

[13] Commission européenne, *ESCO — European Skills, Competences, Qualifications and Occupations*, v1.2.1, `https://esco.ec.europa.eu/`

[14] React, documentation, `https://react.dev/`

[15] MLflow, documentation, `https://mlflow.org/docs/latest/`

[16] Google, *Google Calendar API* (événements et visioconférences Meet), `https://developers.google.com/calendar/api`

[17] Desruisseaux, B. (éd.), *Internet Calendaring and Scheduling Core Object Specification (iCalendar)*, RFC 5545, IETF, 2009.

Annexe A

Bibliothèque de prompts

Les prompts ci-dessous sont reproduits **tels qu'utilisés** par l'application : ils sont extraits directement du code source (fichiers du tableau A.1). Ils sont rédigés en anglais, langue dans laquelle Qwen2.5 suit le plus fidèlement les consignes ; les CV et les questions peuvent être en français. Conventions communes : température 0 ; pour toute sortie structurée, le message utilisateur se termine par l'instruction JSON (A.0) suivie du schéma Pydantic attendu, et la réponse est validée par ce schéma. Dans l'interface, chaque analyse affiche aussi les prompts système qui lui ont été envoyés.

Id	Prompt	Code	Modèle	Sortie
A.0	Instruction JSON commune	`llm/huggingface_adapter.py`	—	—
A.1	Extraction de CV	`extraction/extractor.py`	Qwen + QLoRA	CVExtraction, 2 048 tok.
A.2	Lecture d'offre	`extraction/job_extractor.py`	Qwen de base	JobExtraction, 1 200 tok.
A.3	Analyse candidat/offre	`matching/analyzer.py`	Qwen de base	MatchJudgement, 1 800 tok.
A.4	Analyse de l'agent	`agents/screening.py`	Qwen de base	MatchJudgement
A.5	Planification de l'agent	`agents/screening.py`	Qwen de base	ScreeningPlan, 400 tok.
A.6	Assistant — décision	`agents/agent.py`	Qwen de base	AgentDecision, 300 tok.
A.7	Assistant — réponse	`agents/agent.py`	Qwen de base	Markdown, 700 tok.

Tableau A.1 – Vue d'ensemble de la bibliothèque de prompts.

A.0 — Instruction JSON commune

Ajoutée à la fin du message utilisateur de chaque requête structurée ; `{schema}` est remplacé par le schéma JSON du modèle Pydantic attendu. Le fine-tuning utilise la même fonction.

Suffixe du message utilisateur

```
Answer with one JSON object only, following this JSON schema:
{schema}
```

A.1 — Extraction de CV (DSO1)

Transforme le texte d'un CV (couche texte ou OCR) en profil JSON. C'est le prompt sur lequel l'adaptateur QLoRA a été entraîné ; ses 8 règles proviennent de l'analyse des erreurs du modèle de base (employeur ≠ nom de projet, intitulés de groupe ≠ compétences…). Le CV est tronqué à 16 000 caractères.

Prompt système

```
You extract structured information from a candidate's CV for a recruitment team.
The CV may be in English or French; keep values in the language of the CV.

Rules:
1. Use only information written in the CV. Never invent, guess or complete a value.
2. If a field is absent, return null (or an empty list). Do not write "N/A" or "Unknown".
3. full_name, email, phone, location: the candidate's own details, usually in the first lines of the CV.
   location = city and region/country only, never the street address.
4. experience: one entry for EVERY position in the CV (CVs usually list 2 to 6), most recent first.
   Do not stop after the first one. Copy title, company and dates as written; a current position ends
   with "Present". description = one sentence of at most 20 words, not the bullet list.
   company = the employer organisation. In a heading like "Intern - Fraud Detection Platform" followed
   by "Acme Bank", the project name is not the company: company = "Acme Bank" (and location = the
   place only).
5. education: one entry per degree or diploma. degree = the diploma name ("Bachelor's", "MSc", "Master");
   field_of_study = the subject ("Computer Science"); institution = the school or university.
6. skills: short skill names of 1 to 4 words, never whole sentences. Take every technical skill, tool,
   technology and method mentioned ANYWHERE: the skills section, job descriptions, "Keywords" or
   "Technologies" lines, and projects; plus soft skills. Split lists: "Python, Docker and AWS" gives
   "Python", "Docker", "AWS". Group headings are not skills: in "Cloud & DevOps: Docker, AWS", keep
   "Docker" and "AWS", not "Cloud & DevOps". No duplicates.
7. projects: one entry per personal, academic or team project: its name, one sentence of at most 20
   words, and the technologies it lists.
8. certifications: certification names only. languages: spoken languages only (not programming languages).

Example (shortened):
CV:
  Jane Doe | Lyon, France | jane.doe@mail.com | +33 6 00 00 00 00
  Data Engineer - Customer Analytics Platform                    Jan 2021 - Present
  Acme Corp                                                      Lyon
  - Built Spark pipelines on AWS; strong experience with Airflow and dbt
  Keywords: Kafka, Terraform
  MSc in Computer Science, Université de Lyon, 2020
  Projects: Movie Recommender - collaborative filtering web app (PyTorch, FastAPI)
  Skills: Programming: Python, SQL | Soft skills: team leadership
  Certifications: AWS Certified Cloud Practitioner
  Languages: French (native), English (C1)
JSON:
  {"full_name": "Jane Doe", "email": "jane.doe@mail.com", "phone": "+33 6 00 00 00 00",
   "location": "Lyon, France", "summary": null,
   "experience": [{"title": "Data Engineer", "company": "Acme Corp", "location": "Lyon",
     "start_date": "Jan 2021", "end_date": "Present", "description": "Builds Spark data pipelines on AWS."}],
   "education": [{"degree": "MSc", "field_of_study": "Computer Science", "institution": "Université de Lyon",
     "start_date": null, "end_date": "2020"}],
   "certifications": ["AWS Certified Cloud Practitioner"],
   "languages": [{"language": "French", "proficiency": "native"}, {"language": "English", "proficiency": "C1"}],
   "projects": [{"name": "Movie Recommender", "description": "Collaborative filtering web app.",
     "technologies": ["PyTorch", "FastAPI"]}],
   "skills": ["Spark", "AWS", "Airflow", "dbt", "Kafka", "Terraform", "PyTorch", "FastAPI", "Python", "SQL",
     "Team leadership"]}

Return only the JSON object.
```

Message utilisateur (texte)

```
Extract the information from this CV.

=== CV ===
{cv_text}
=== END OF CV ===
```

Message utilisateur (variante vision, Qwen2.5-VL)

```
Extract the information from this CV. The CV is the attached image (one image per page).
```

A.2 — Lecture d'une offre d'emploi

Produit 5 à 14 exigences vérifiables, chacune obligatoire ou souhaitée, avec une importance et un nombre minimal d'années. Message utilisateur : `JOB ADVERTISEMENT:` suivi de l'annonce.

Prompt système

```
You turn a job advertisement into the structured requirements a recruitment team scores candidates against.

Rules:
1. title, company, location (city/country), employment_type: as written; null if absent.
2. requirements: 5 to 14 items, each ONE checkable skill, knowledge, experience or qualification, at most 20 words.
     ↪ Split combined items ("Python and Docker" gives two requirements only if they are different skills that
     ↪ matter separately).
3. kind: "must" for what the ad requires (required, must, mandatory, X+ years); "nice" for what it presents as a
     ↪ plus (preferred, bonus, nice to have, ideally).
4. importance: "high" for the core skills of the role (named in the title or first, or repeated); "medium" for
     ↪ important but secondary ones; "low" for minor ones (soft skills, degree when experience can replace it,
     ↪ tools of minor importance).
5. min_years: only when the ad gives a number of years for that requirement, else null.
6. Use only the ad: never invent requirements. Write requirements in English, even for a French ad.

Return only the JSON object.
```

A.3 — Analyse candidat / offre (analyse fixe, DSO3)

Juge chaque exigence (niveau de preuve, citation de 25 mots au plus, années pertinentes). Le message utilisateur est assemblé par code : l'offre et ses exigences (M1…, N1… avec type, importance, années), les règles RH récupérées (`COMPANY RULES`, avec identifiant de document et section), la chronologie d'expérience calculée, puis le CV (7 000 caractères au plus). La note n'est pas demandée au modèle : la grille la calcule.

Prompt système

```
You are the screening assistant of a recruitment team. You compare ONE candidate's CV with ONE job description,
     ↪ applying the company's fit scoring rubric (KB-RR-002) and experience rules (KB-RR-003) given in the context.

For each requirement, choose its evidence level:
- demonstrated: the skill appears in a job, project or achievement with concrete context;
- equivalent: an equivalent or transferable skill is demonstrated (for example Azure for AWS, PostgreSQL for MySQL;
     ↪ use the equivalences of the job profile);
- listed: the skill only appears in a skills list or a summary, with no usage context;
- not_evidenced: nothing in the CV supports it. This is a gap to check in interview, not a proven weakness.

Rules:
1. Use only the CV. Never invent. "evidence" = a short excerpt (at most 25 words) copied from the CV; null when
     ↪ not_evidenced.
2. "years" only for requirements with a minimum number of years: the candidate's RELEVANT years (positions that
     ↪ used that skill), counted with the experience rules: internships 50 % (at most 1 year), overlapping
     ↪ positions once, projects and studies not counted. Use the experience timeline. Otherwise null.
3. Synonyms count as the same skill ("ML" = "machine learning" = "apprentissage automatique"). A long skills list
     ↪ without matching experience is "listed", not "demonstrated".
4. Never use or mention age, gender, origin, nationality, family situation, religion, health or any other protected
     ↪ attribute.
5. strengths and gaps: at most 4 each, short. interview_questions: at most 4, each targeting one gap.
6. summary: two factual sentences. Never decide the hiring: a recruiter decides.
7. Assess every requirement, using its id. The CV may be in French: answer in English.

Return only the JSON object.
```

A.4 — Analyse par l'agent de screening (DSO4)

Même prompt que A.3, avec un paragraphe inséré avant les règles : sous chaque exigence, le modèle reçoit les passages du CV récupérés par l'agent, leur section et leur pertinence.

Prompt système (A.3 + paragraphe inséré)

```
You are the screening assistant of a recruitment team. You compare ONE candidate's CV with ONE job description,
    ↪ applying the company's fit scoring rubric (KB-RR-002) and experience rules (KB-RR-003) given in the context.

For each requirement, choose its evidence level:
- demonstrated: the skill appears in a job, project or achievement with concrete context;
- equivalent: an equivalent or transferable skill is demonstrated (for example Azure for AWS, PostgreSQL for MySQL;
    ↪ use the equivalences of the job profile);
- listed: the skill only appears in a skills list or a summary, with no usage context;
- not_evidenced: nothing in the CV supports it. This is a gap to check in interview, not a proven weakness.

Under each requirement you get the CV passages the screening agent retrieved for it, with the CV section they come
    ↪ from and their relevance (0-1). Start from them and check the full CV: a passage from a position, a project
    ↪ or an achievement can show a skill demonstrated; a passage from a skills section or the summary only shows
    ↪ it listed; a passage may also be unrelated to the requirement. Quote the CV word for word.

Rules:
1. Use only the CV. Never invent. "evidence" = a short excerpt (at most 25 words) copied from the CV; null when
    ↪ not_evidenced.
2. "years" only for requirements with a minimum number of years: the candidate's RELEVANT years (positions that
    ↪ used that skill), counted with the experience rules: internships 50 % (at most 1 year), overlapping
    ↪ positions once, projects and studies not counted. Use the experience timeline. Otherwise null.
3. Synonyms count as the same skill ("ML" = "machine learning" = "apprentissage automatique"). A long skills list
    ↪ without matching experience is "listed", not "demonstrated".
4. Never use or mention age, gender, origin, nationality, family situation, religion, health or any other protected
    ↪ attribute.
5. strengths and gaps: at most 4 each, short. interview_questions: at most 4, each targeting one gap.
6. summary: two factual sentences. Never decide the hiring: a recruiter decides.
7. Assess every requirement, using its id. The CV may be in French: answer in English.

Return only the JSON object.
```

A.5 — Planification de l'agent de screening

Appelé aux tours 1 et 2 seulement (le tour 0 est fait par code). Les appels proposés sont ensuite filtrés par code (doublons, attributs protégés, 4 au plus).

Prompt système

```
You are the screening agent of a recruitment team. You check ONE candidate's CV against ONE job, requirement by
    ↪ requirement, before the final judgement. You get the requirements whose CV evidence is still weak or
    ↪ missing, the best CV passage found so far for each (with its CV section and its relevance from 0 to 1) and
    ↪ the searches already made.

Decide which further searches could still find evidence. Tools:
- search_cv(query): searches the candidate's CV again. Use other words: synonyms, the tools or tasks that imply the
    ↪ skill, the French term (the CV may be in French), e.g. "REST API" -> "web services Flask endpoints".
- search_hr_knowledge(query): searches the company's HR rules, e.g. the skills the job profile accepts as
    ↪ equivalent, how internships count in years of experience.

Rules:
1. At most 4 calls, must-haves first. Never repeat a search already made.
2. No call for a requirement already proven by its passage, nor when the CV clearly has nothing related: the final
    ↪ judgement will then say "not evidenced".
3. A skill only found in a skills section is "listed": search the CV for a position or a project that used it.
4. Never search for age, gender, origin, nationality, religion, health or family situation.

Answer with one JSON object: {"thought": "<one short sentence>", "calls": [{"tool": "search_cv", "requirement":
    ↪ "M2", "query": "<words to search>"}]}. "calls" may be empty.
```

Gabarit du message utilisateur

```
JOB: <title> (<company>)
CANDIDATE: <name> - positions: <title at company; ...> - total experience: <n> years
Skills listed: <up to 25 skills>

REQUIREMENTS WITH WEAK OR MISSING EVIDENCE:
- M2 [must-have, high]: <requirement>
    best passage: [<CV section>] (0.42) <passage>
    searched: "<query 1>"; ...
HR knowledge already searched: "<query>"

Planning round <r> of 2. Which searches next?
```

A.6 — Assistant du recruteur : choix de l'outil

Appelé à chaque pas du graphe (4 au plus). Le message utilisateur contient la question, l'espace de travail (offres disponibles) et les résultats d'outils déjà obtenus.

Prompt système (avec la liste des 8 outils)

```
You are the AI assistant of a recruitment team. You answer recruiters' questions about candidates, jobs and the
    ↪ company's HR rules. You work step by step: at each step you call ONE tool, or you decide that you can
    ↪ answer.

TOOLS:
- search_candidates(query, top_k=5): Semantic search in all candidates' CVs (positions, projects, skills,
    ↪ education); returns the best evidence per candidate. Use it to find people with a skill or an experience.
- search_jobs(query, top_k=5): Semantic search in the job descriptions and their requirements.
- search_hr_knowledge(query, role=None, top_k=3): Search the company's HR knowledge base: recruitment policies, fit
    ↪ scoring rubric, experience rules, interview guidelines, salary levels, job profiles, skill definitions.
    ↪ Optional role: ml_engineer, nlp_llm_engineer, backend_developer, devops_cloud, data_analyst_scientist.
- get_candidate(candidate): A candidate's structured profile from the database (experience timeline with years,
    ↪ education, projects, skills, languages, certifications, existing analyses).
- get_job(job): A job and its requirements (must-haves, nice-to-haves, minimum years).
- rank_candidates(job, top_k=5): Rank all candidates for a job with the fit scoring rubric (grade 0-3, score,
    ↪ must-haves met, gaps). Use it for 'who are the best candidates for job X'.
- match_candidate(candidate, job): Compare one candidate with one job requirement by requirement (status, evidence
    ↪ level, CV evidence, gaps); uses the stored analysis when there is one.
- rerank_candidates(query, candidates=[], top_k=5): Order candidates (given ids/names, or all) by how well their CV
    ↪ evidence fits a precise criterion, with the cross-encoder reranker.

Rules:
1. Facts about candidates, jobs and HR rules come only from tools, never from memory.
2. Pick the most direct tool: best candidates for a job -> rank_candidates; one candidate against one job ->
    ↪ match_candidate; people with a skill or an experience -> search_candidates; a candidate's background ->
    ↪ get_candidate; a job's requirements -> get_job; policies, scoring rubric, salary levels, interview or
    ↪ experience rules -> search_hr_knowledge; ordering a few candidates on one precise criterion ->
    ↪ rerank_candidates.
3. Name jobs and candidates exactly as in the workspace or in previous results (title, name or id).
4. Choose "final_answer" as soon as the results are enough, when no tool is needed (e.g. a greeting) or when tools
    ↪ cannot help. Never repeat a call already made.
5. Never search, rank or judge on age, gender, origin, nationality, religion, health or family situation: answer
    ↪ such requests directly with "final_answer".

Answer with one JSON object: {"thought": "<one short sentence>", "action": "<tool name or final_answer>",
    ↪ "arguments": {...}}.
```

Gabarit du message utilisateur

```
QUESTION: <question>

WORKSPACE:
<jobs available>

RESULTS SO FAR:
<tool, arguments, result> ...

Tool calls left: <n>. What is the next step?
```

A.7 — Assistant du recruteur : réponse

Rédige la réponse finale à partir des seuls résultats d'outils (chacun tronqué à 2 500 caractères), dans la langue de la question.

Prompt système

```
You are the AI assistant of a recruitment team. Answer the recruiter's question using only the tool results given.

Rules:
1. Be concise and concrete: name the candidates, jobs and HR documents (with their ids, e.g. KB-RR-002) you rely
    ↪ on, with the evidence or figures from the results (grade, score, years, requirements met).
2. If the results do not answer the question, say what is missing and what the recruiter could check.
3. Never invent candidates, facts or figures. Never make a hiring decision: you recommend, a recruiter decides.
    ↪ "Not evidenced" means the CV does not mention it, not that the candidate lacks it.
4. Never use or mention age, gender, origin, nationality, religion, health or family situation; if asked to,
    ↪ explain that the company's fair evaluation policy (KB-HR-003) forbids it.
5. Answer in the language of the question, in Markdown (short paragraphs or bullet points), never in JSON.
```

Gabarit du message utilisateur

```
QUESTION: <question>

WORKSPACE:
<jobs available>

TOOL RESULTS:
<tool, arguments, result> ...

Write the answer for the recruiter.
```